

## Department of Computer Science and Engineering

### Assignment Questions

| Particular             | Details                                                                                                                                                                                                                                                                                                                                                                 |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Course Code & Name     | CSA0312 – Data Structures – Slot B                                                                                                                                                                                                                                                                                                                                      |
| Assignment Title       | Design Intelligent Emergency Evacuation Routing System for a city containing more than 8000 intersections and 20,000 weighted roads                                                                                                                                                                                                                                     |
| Course Outcome(s) – CO | CO2: Apply the suitable Abstract Data type for construction of the smart city emergency system.<br>CO4: Make use of heaps, priority queue and hashing techniques for graph storage and route cache for repeated queries in C.<br>CO5: Develop robust smart city emergency system by implementing shortest path algorithms such as Dijkstra's algorithm, Floyd-Warshall. |
| Bloom's Taxonomy Level | L4 – Analyze                                                                                                                                                                                                                                                                                                                                                            |
| SDG Mapping (SDG 1-17) | SDG 9 - Industry, Innovation, and Infrastructure<br>SDG 11 – Sustainable Cities and Communities<br>SDG 13 – Climate Action                                                                                                                                                                                                                                              |

#### Problem Statement:

A smart-city disaster management authority is developing an Intelligent Emergency Evacuation Routing System for a city containing more than 8000 intersections and 20,000 weighted roads. During floods, earthquakes, industrial accidents, or large-scale fires, road travel costs change dynamically because of congestion, road blockages, damaged infrastructure, and emergency restrictions.

The system receives continuous updates containing intersections, roads, travel costs, road status, evacuation centers, and emergency vehicle locations. It must identify the safest and shortest available route from multiple disaster locations to designated evacuation centers while operating under the following constraints:

- The network contains 8000+ vertices and 20,000+ weighted edges.
- Road weights change frequently because of disaster conditions.
- Blocked roads must be removed or temporarily disabled.
- The system should support multiple source-to-destination queries.
- Repeated evacuation queries should be answered efficiently.
- Memory consumption should remain within moderate computational resources.
- The system should avoid repeatedly processing the same intersections.

- The routing system should prioritize routes with minimum travel cost.
- The system should identify unreachable evacuation centers.
- The implementation must be developed in C.

The engineering team is considering Adjacency Matrix, HashMap-based Adjacency List, HashSet, Min-Heap, Priority Queue, Dijkstra's Algorithm, Floyd-Warshall, and route caching.

**Students must:**

1. Analyze and justify the choice of graph ADT and graph representation.
2. Design suitable C structures for vertices, edges, hash entries, heap nodes, and routes.
3. Implement dynamic road-weight and road-status updates.
4. Use a Min-Heap to optimize Dijkstra's algorithm.
5. Use a HashSet to track processed intersections.
6. Implement route reconstruction.
7. Implement route caching for repeated queries.
8. Handle blocked roads, invalid vertices, duplicate roads, and unreachable destinations.
9. Compare Dijkstra's algorithm with Floyd-Warshall for the given network.
10. Analyze the time and space complexity of the complete system.
11. Evaluate whether the design can satisfy a 200 ms routing requirement.
12. Analyze how the system behaves when 30% of the roads become unavailable during a disaster.

**ANSWER**

| Student Name | Register Number | Team Members                                                    | Date       |
|--------------|-----------------|-----------------------------------------------------------------|------------|
| Athisaya U   | 192571001       | MONIKA B (192571002)<br>PAPAGARI PUSHPA<br>CHARITHA (192525371) | 31/08/2026 |

**GitHub Repository:** <https://github.com/athisayaprema2007-hue/CSA0312-Intelligent-Evacuation-Routing-System>

## A. Student Work

### 1. Problem Understanding and Formulation

#### What is the problem?

The city road network must support emergency evacuation routing while road conditions are changing. Intersections are vertices and bidirectional roads are weighted edges. During disasters, road weights may increase because of congestion or damage, roads may be blocked or reopened, and multiple disaster locations may request routes to evacuation centres. The system therefore needs to find the minimum-cost available route on a large, sparse and dynamically changing graph while reporting invalid or unreachable destinations safely.

#### Expected Outcomes

- Correct minimum-cost routes for valid source-to-destination queries.
- Dynamic handling of road-weight changes, blocking and reopening.
- Efficient repeated-query handling through version-aware route caching.
- Explicit detection of invalid vertices, duplicate roads, negative weights and unreachable destinations.
- Performance comfortably below the 200 ms routing requirement on the verified test configuration.
- Graceful behaviour when 30% of roads become unavailable.

#### Available Information / Data

The assignment specifies a road network of more than 8,000 vertices and 20,000 weighted edges, continuous road-cost/status updates, multiple route queries and a C implementation. For verification, the program generates a deterministic synthetic city with 8,200 intersections and 21,000 logical bidirectional roads (42,000 directed edge records), using seed 20260831. Functional tests use a small hand-verifiable graph and the full-scale benchmark uses ten fixed source-to-destination queries.

#### Assumptions

- Road weights are non-negative so Dijkstra's algorithm is valid.
- Each logical city road is bidirectional and is represented internally by two directed edge records.
- Blocked roads remain stored but are marked inactive so they can later be reopened without data loss.
- The full-scale city is deterministic synthetic data for course evaluation; no real-world deployment or live GPS/traffic validation is claimed.
- One scalar travel cost represents the combined routing cost used by the prototype.

## Constraints to be Satisfied

- 8,000+ vertices and 20,000+ weighted edges.
- Frequent road-weight and road-status updates.
- Multiple and repeated source-to-destination queries.
- Moderate memory use suitable for a sparse graph.
- No repeated finalization of processed intersections in cyclic graphs.
- Minimum travel cost, unreachable-centre detection and robust validation.
- Implementation in C and evaluation against the 200 ms target.
- Analysis of behaviour after 30% of roads become unavailable.

## 2. Application of Course Knowledge

The solution applies graph ADTs, hashing, heaps, priority queues and shortest-path algorithms as an integrated data-structures design. The graph is sparse, so a HashMap-based adjacency list is selected instead of an adjacency matrix. A binary Min-Heap implements the Dijkstra priority queue, a HashSet tracks finalized intersections, predecessor arrays support route reconstruction, and a hash-based route cache accelerates repeated queries.

### Graph Representation Selection

| Representation         | Time / Space Effect                                | Decision               |
|------------------------|----------------------------------------------------|------------------------|
| Adjacency Matrix       | $O(V^2)$ memory; $O(V)$ neighbour scan             | Rejected               |
| HashMap Adjacency List | $O(V+E)$ memory; $O(\text{degree})$ neighbour scan | Selected               |
| Floyd-Warshall Matrix  | $O(V^3)$ preprocessing and $O(V^2)$ memory         | Rejected at full scale |

With 8,200 intersections, the number of possible undirected pairs exceeds 33 million, while the verified network contains only 21,000 logical roads. The adjacency list therefore avoids large matrix waste and lets Dijkstra scan only existing neighbours.

### C Structure Design

| Structure    | Important Fields                    | Purpose                              |
|--------------|-------------------------------------|--------------------------------------|
| Edge         | dest_id, weight, active, next       | One directed half of a road          |
| Vertex       | id, index, edges, degree            | Intersection and adjacency-list head |
| Graph        | vertices, by_index, counts, version | Complete mutable road network        |
| HashMapEntry | key, value, next                    | ID-to-Vertex lookup                  |

| Structure          | Important Fields                   | Purpose                           |
|--------------------|------------------------------------|-----------------------------------|
| HashSetEntry       | key, next                          | Processed-intersection membership |
| HeapNode / MinHeap | vertex_index, dist, nodes, pos     | Priority queue and decrease-key   |
| Route              | IDs, length, total_cost, reachable | Reconstructed query result        |
| RouteCacheEntry    | src, dst, graph_version, route     | Versioned repeated-query result   |

```

typedef struct Edge {
    int dest_id;
    long weight;
    int active;
    struct Edge *next;
} Edge;

typedef struct Route {
    int src_id, dst_id;
    int *intersection_ids;
    int num_intersections;
    long total_cost;
    int reachable;
} Route;

```

### Key Algorithmic Reasoning

The Min-Heap provides  $O(\log V)$  insert, extract-min and decrease-key operations. A reverse `pos[vertex_index]` array locates heap nodes in  $O(1)$  before the logarithmic heap repair. The HashSet keeps average add/contains operations  $O(1)$  and guarantees that cyclic graphs do not repeatedly finalize the same vertex. With an adjacency list and binary heap, an uncached Dijkstra query runs in  $O((V+E) \log V)$  time with  $O(V)$  working memory in addition to  $O(V+E)$  graph storage.

### 3. Solution / Design / Methodology

The implementation is modular. Road updates mutate the graph while preserving consistency in both directions. Every routing-relevant mutation increments `Graph.version`. Dijkstra ignores inactive edges, the HashSet finalizes each intersection once, the predecessor array reconstructs the route, and the route cache accepts a cached answer only when its graph version matches the current graph.

## System Architecture Diagram

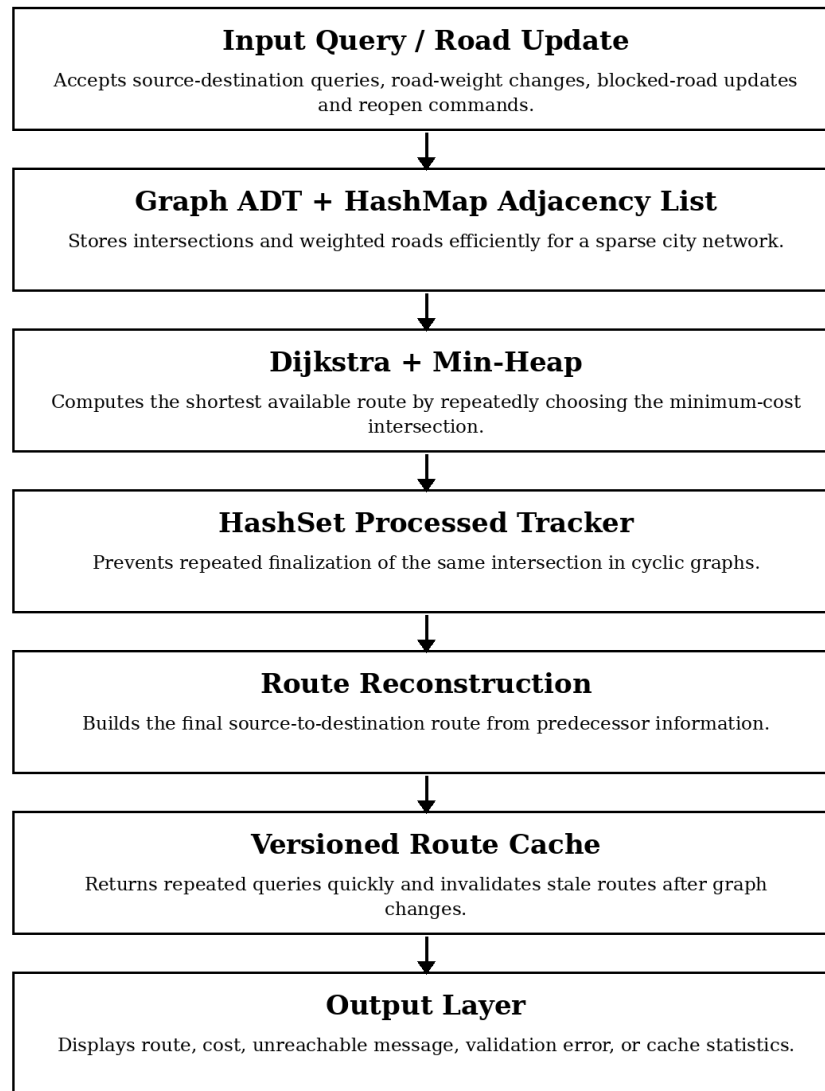


Figure 1. System architecture of the Intelligent Emergency Evacuation Routing System.

### Dynamic Road-Weight and Road-Status Updates

Each logical road is stored as two directed Edge records. `graph_update_road_weight` validates both endpoints, rejects negative values, locates both edge records and updates both weights together. `graph_set_road_status` changes both active flags. A blocked road remains stored with `active = 0` so reopening is symmetric. Every routing-relevant mutation increments `Graph.version` because a previously cached route may now be stale.

```

UpdateRoadWeight(G, u, v, w):
    if u or v is invalid: return INVALID_VERTEX
    if w < 0: return NEGATIVE_WEIGHT
    locate edge u->v and edge v->u
    if either edge is absent: return NO_SUCH_ROAD
    update both weights
    G.version = G.version + 1
    return OK

SetRoadStatus(G, u, v, active):
    validate endpoints and locate both edge records
    set both active flags
    G.version = G.version + 1
    return OK

```

## Min-Heap Integration

Insert appends a node and sifts upward. Extract-min removes the root, moves the last node to the root and heapifies downward. Decrease-key lowers a tentative distance and sifts upward. Each heap swap updates the reverse position map.

```

ExtractMin(H):
    minimum = H.nodes[0]
    move H.nodes[H.size - 1] to the root
    reduce H.size and repair pos[]
    Heapify(H, 0)
    return minimum

DecreaseKey(H, vertex, new_distance):
    i = H.pos[vertex]
    reject an absent vertex or a non-decreasing value
    H.nodes[i].dist = new_distance
    SiftUp(H, i)

```

## Dijkstra and Route Reconstruction

The source and destination are resolved through the graph HashMap. dist[] is initialized to infinity, prev[] to NONE and the source distance to zero. The algorithm repeatedly extracts the minimum tentative distance, finalizes that vertex through the HashSet and relaxes only active outgoing roads. With non-negative weights, the first extraction of the destination proves the shortest distance.

```

Dijkstra(G, src, dst):
    dist[*] = INF; prev[*] = NONE; dist[src] = 0
    insert src into the MinHeap
    processed = empty HashSet
    while heap is not empty:
        u = ExtractMin(heap)
        if u is in processed: continue
        add u to processed
        if u == dst: break
        for every active edge (u, v, w):
            if v is not processed and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                prev[v] = u
                decrease-key or insert v
    return dist and prev

ReconstructRoute(src, dst, dist, prev):
    if dist[dst] == INF: return UNREACHABLE
    count nodes while following prev[] from dst
    allocate route IDs
    fill IDs from the last index down to zero
    return route with total_cost = dist[dst]

```

## Versioned Route Cache

RouteCache is a hash table keyed by the ordered pair (source, destination). Each entry stores a deep copy of the Route and the graph version that produced it. A matching version is an average  $O(1)$  cache hit. A different version means that the graph changed; the stale entry is removed and the query is recomputed. Unreachable results are cached as well.

```

EvacQuery(G, cache, src, dst):
    validate source and destination
    route = CacheLookup(cache, src, dst, G.version)
    if route exists: return route
    route = Dijkstra plus route reconstruction
    CacheStore(cache, route, G.version)
    return route

```

## Exceptional and Failure Handling

| Condition                     | Required Behaviour                    | Verification    |
|-------------------------------|---------------------------------------|-----------------|
| Invalid source or destination | Return NULL / INVALID before search   | Tests 6 and 7   |
| Duplicate road                | Reject without changing the graph     | Test 8          |
| Negative weight               | Return<br>GRAPH_ERR_NEGATIVE_WEIGHT   | Test 9          |
| Blocked road                  | Keep the edge; skip active = 0        | Test 4          |
| Reopened road                 | Restore traversal using stored weight | Test 5          |
| Missing road                  | Return<br>GRAPH_ERR_NO_SUCH_ROAD      | Graph module    |
| Unreachable destination       | reachable = 0 and cost = -1           | Tests 10 and 16 |
| Cyclic graph                  | HashSet finalizes each vertex once    | Test 15         |



Every allocation is checked. Cleanup functions release graphs, maps, sets, heaps, routes and cache entries. Partial road insertion is rolled back if the second directed edge cannot be allocated.

#### 4. Use of Modern Tools

- GCC compiler for building the modular C implementation with optimization and warning checks.
- Windows Command Prompt for running the guided demo, functional tests, benchmark and road-failure experiment.
- GitHub for version control and submission hosting.
- Microsoft Word and PDF export for preparing the technical report.
- Deterministic result files in results/ (test\_results.txt, benchmark\_results.csv and road\_failure\_analysis.txt) so reported values can be independently re-checked from the submitted project.

Tool usage is evidenced by the Windows terminal screenshots below, the generated result files, and the public GitHub repository containing the modular implementation and report.

#### 5. Results and Validation

##### Functional Test Results

| Test Group       | Coverage                                     | Result       |
|------------------|----------------------------------------------|--------------|
| Shortest route   | Hand-verified path and cost                  | PASS         |
| Dynamic graph    | Weight update, block and reopen              | PASS         |
| Validation       | Invalid IDs, duplicates and negative weights | PASS         |
| Reachability     | Disconnected and isolated vertices           | PASS         |
| Repeated queries | Cache miss, hit and invalidation             | PASS         |
| Cycles           | No repeated finalization                     | PASS         |
| Overall          | 16 functional tests                          | 16 / 16 PASS |

##### 200 ms Performance Validation

The deterministic benchmark uses 8,200 intersections and 21,000 logical bidirectional roads, represented internally by 42,000 directed edge records. Ten fixed source-to-destination queries were executed. The measured uncached average was approximately 3.91 ms per query, the worst observed uncached query was 7.000 ms, and cached hits averaged approximately 0.00003 ms. The measured configuration was Windows 11 on an Intel Core Ultra 7 255H system with 31.4 GB usable RAM, using GCC 3.4.2 with -O2 optimisation.

| Metric                 | Measured Result          |
|------------------------|--------------------------|
| Intersections          | 8,200                    |
| Logical roads          | 21,000                   |
| Directed edge records  | 42,000                   |
| Average uncached query | Approximately 3.91 ms    |
| Worst uncached query   | 7.000 ms                 |
| Average cached hit     | Approximately 0.00003 ms |
| Functional tests       | 16 / 16 PASS             |

The verified configuration therefore satisfies the 200 ms routing requirement with substantial margin. This conclusion applies to the stated machine, compiler, deterministic data and clock resolution and is not claimed as a universal hardware guarantee.

### 30% Road-Failure Validation

The failure experiment blocks exactly 6,300 of the 21,000 logical roads using a deterministic shuffle. All ten selected source-destination queries remain reachable, but global reachability from the reference intersection falls from 8,200/8,200 to 8,026/8,200. Therefore 174 intersections, approximately 2.1%, become disconnected from the main reachable region.

| Observation                      | Before Failure | After 30% Failure |
|----------------------------------|----------------|-------------------|
| Open logical roads               | 21,000         | 14,700            |
| Globally reachable intersections | 8,200          | 8,026             |
| Isolated from reference source   | 0              | 174               |
| Selected destinations reachable  | 10 / 10        | 10 / 10           |
| Cached routes invalidated        | 0              | 10                |
| Route-cost change                | Baseline       | +3 to +175        |

Route lengths usually increase because blocked segments force detours. The graph version changes after blocking, so all ten previously cached routes are recognized as stale and recomputed. The experiment demonstrates graceful degradation: most of the city remains routable while isolated neighbourhoods are explicitly identified.

### Verified Execution Evidence

The following screenshots were captured from the Windows terminal while running the verified program. Timing values can vary slightly between runs because of clock resolution, but all uncached queries remain far below the required 200 ms limit.

```
Command Prompt

[8] Multiple queries from different disaster sites.
Query 2 -> 7 [cache MISS, Dijkstra run]: Route: 2 -> 4 -> 5 -> 6 -> 7 (total travel cost 11, 5 intersections)
Query 3 -> 6 [cache MISS, Dijkstra run]: Route: 3 -> 1 -> 2 -> 4 -> 5 -> 6 (total travel cost 15, 6 intersections)
Query 7 -> 1 [cache MISS, Dijkstra run]: Route: 7 -> 6 -> 5 -> 4 -> 2 -> 1 (total travel cost 15, 6 intersections)

[9] Unreachable center: intersection 8 has no roads.
Query 1 -> 8 [cache MISS, Dijkstra run]: Destination 8 is UNREACHABLE from 1 over active roads.

[10] Invalid queries.
Query 99 -> 4 [cache MISS, Dijkstra run]: rejected: invalid intersection ID
Query 1 -> -5 [cache MISS, Dijkstra run]: rejected: invalid intersection ID

[11] Cache statistics: hits=1 misses=8 stale_evictions=3

Demo finished; freeing all memory.
test_runner: 16/16 tests passed

C:\Users\jimin\OneDrive\Desktop\CSA0312 ASSIGNMENT\CSA0312_clean_submission>
```

Figure 2. Windows terminal screenshot showing multiple routes, unreachable and invalid-query handling, cache statistics, and 16/16 functional tests passed.

```

Command Prompt
Generating deterministic city (seed 20260831): 8200 intersections, 21000 logical roads...
Graph ready: 8200 vertices, 21000 logical roads (42000 directed edges).
Query 1: 4546 -> 2343 cost= 161 hops= 9 uncached avg 1.200 ms c
ached avg 0.000000 ms
Query 2: 488 -> 4320 cost= 154 hops= 5 uncached avg 0.867 ms c
ached avg 0.000010 ms
Query 3: 7680 -> 4213 cost= 212 hops= 9 uncached avg 2.333 ms c
ached avg 0.000030 ms
Query 4: 8005 -> 1947 cost= 223 hops= 11 uncached avg 1.800 ms c
ached avg 0.000010 ms
Query 5: 5674 -> 1101 cost= 241 hops= 7 uncached avg 2.533 ms c
ached avg 0.000000 ms
Query 6: 3568 -> 5188 cost= 215 hops= 12 uncached avg 6.400 ms c
ached avg 0.000020 ms
Query 7: 7204 -> 7818 cost= 238 hops= 13 uncached avg 3.333 ms c
ached avg 0.000000 ms
Query 8: 7116 -> 6482 cost= 228 hops= 9 uncached avg 2.733 ms c
ached avg 0.000010 ms
Query 9: 4541 -> 3854 cost= 241 hops= 10 uncached avg 2.133 ms c
ached avg 0.000020 ms
Query 10: 6012 -> 4594 cost= 265 hops= 7 uncached avg 3.467 ms c
ached avg 0.000020 ms
Benchmark written to results\benchmark_results.csv
Cache stats: hits=1000000 misses=10 stale_evictions=0

C:\Users\jimin\OneDrive\Desktop\CSA0312 ASSIGNMENT\CSA0312_clean_submission>

```

Figure 3. Windows terminal screenshot from the full-scale benchmark with 8,200 intersections, 21,000 logical roads and ten timed route queries.

```

Command Prompt
Building the same deterministic city as the benchmark...
Blocked 6300 of 21000 logical roads (30%).
Analysis written to results\road_failure_analysis.txt

C:\Users\jimin\OneDrive\Desktop\CSA0312 ASSIGNMENT\CSA0312_clean_submission>

```

Figure 4. Windows terminal screenshot from the road-failure experiment, where exactly 6,300 of 21,000 logical roads were blocked and the analysis file was written.

## Requirement Traceability Table

Each assignment requirement is mapped to the section that addresses it and the evidence that validates it.

| Requirement                              | Where Addressed | Evidence                                                 |
|------------------------------------------|-----------------|----------------------------------------------------------|
| 8000+ vertices and 20000+ weighted roads | Section 5       | Figure 3; results/benchmark_results.csv (8,200 / 21,000) |

| Requirement                                 | Where Addressed  | Evidence                                                                                  |
|---------------------------------------------|------------------|-------------------------------------------------------------------------------------------|
| Dynamic road-weight updates                 | Section 3        | Test 3 in results/test_results.txt                                                        |
| Blocked and reopened roads                  | Section 3        | Tests 4 and 5; Figure 2 demo evidence                                                     |
| Multiple source-to-destination queries      | Sections 3 and 5 | Figure 2; ten timed queries in Figure 3                                                   |
| Repeated-query caching                      | Section 3        | Tests 12-14; cache statistics in Figures 2 and 3                                          |
| Moderate memory via adjacency list          | Sections 2 and 6 | $O(V+E)$ storage analysis; representation comparison                                      |
| HashSet tracking of processed intersections | Sections 2 and 3 | Test 15 (cyclic graph, no repeated finalization)                                          |
| Minimum-cost routing                        | Sections 2 and 3 | Tests 1 and 2 (hand-verified route and cost)                                              |
| Unreachable destination handling            | Sections 3 and 5 | Tests 10 and 16; Figure 2                                                                 |
| Implementation in C                         | Sections 2 and 3 | src/ modules: graph.c, hashmap.c, hashset.c, minheap.c, dijkstra.c, route_cache.c, main.c |
| Dijkstra versus Floyd-Warshall comparison   | Section 6        | Comparison table; $5.5 \times 10^{11}$ operation estimate                                 |
| 200 ms routing evaluation                   | Section 5        | 3.91 ms average and 7.000 ms worst uncached query                                         |
| 30% road-failure analysis                   | Section 5        | Figure 4; results/road_failure_analysis.txt (6,300 blocked; 8,026 reachable)              |

## 6. Analysis and Engineering Decision

### Comparison of Alternatives

| Aspect         | Dijkstra with Binary Heap       | Floyd-Warshall                         |
|----------------|---------------------------------|----------------------------------------|
| Problem solved | Single-source / single-pair     | All-pairs                              |
| Time           | $O((V+E) \log V)$ per query     | $O(V^3)$                               |
| Space          | $O(V+E)$ graph + $O(V)$ working | $O(V^2)$ matrix                        |
| Sparse graph   | Uses only existing edges        | Ignores sparsity                       |
| Road update    | Next query reads updated graph  | Precomputed matrix becomes stale       |
| Repeated query | Average $O(1)$ cache hit        | $O(1)$ only after costly preprocessing |
| Decision       | Selected                        | Unsuitable at full scale               |

At  $V = 8,200$ , Floyd-Warshall requires approximately  $5.5 \times 10^{11}$  update operations and about 67.2 million distance cells. Important road changes would also invalidate an all-pairs result. For a sparse, dynamic network with individual route requests, Dijkstra with a binary heap is therefore the appropriate engineering choice.

### Time and Space Complexity

| Operation                | Time Complexity | Space Complexity  |
|--------------------------|-----------------|-------------------|
| HashMap / HashSet lookup | $O(1)$ average  | $O(V)$ total      |
| Add intersection         | $O(1)$ average  | $O(1)$ additional |

| Operation                            | Time Complexity    | Space Complexity                |
|--------------------------------------|--------------------|---------------------------------|
| Add / update / block road            | $O(\text{degree})$ | $O(1)$ additional               |
| Heap insert / extract / decrease-key | $O(\log V)$        | $O(V)$ heap total               |
| Uncached Dijkstra                    | $O((V+E) \log V)$  | $O(V)$ working                  |
| Route reconstruction                 | $O(L)$             | $O(L)$                          |
| Cache lookup / store                 | $O(1)$ average     | $O(C \times \text{average } L)$ |
| Complete graph storage               | -                  | $O(V+E)$                        |

$V$  denotes intersections,  $E$  logical roads,  $L$  route length and  $C$  cached routes. The complete system uses  $O(V+E+C \times \text{average } L)$  memory, which is suitable for moderate computational resources.

### Advantages, Limitations and Trade-offs

- Adjacency-list storage scales with existing roads rather than all possible vertex pairs, but neighbour updates require searching the local adjacency list.
- The Min-Heap improves Dijkstra frontier operations, while the additional `pos[]` array uses  $O(V)$  memory to support efficient decrease-key.
- The HashSet prevents repeated finalization in cyclic graphs at the cost of an additional  $O(V)$  membership structure.
- Global graph-version cache invalidation is simple and correct, but it is conservative because a change on one road can invalidate cached routes that would not actually use that road.
- The benchmark uses deterministic synthetic data and a single scalar travel cost, so the prototype does not claim real-world traffic, safety or GPS validation.
- The implementation is single-threaded and `clock()` has roughly millisecond-scale resolution, limiting precision for extremely fast cached operations.

### Final Engineering Decision

The final design combines a HashMap-based adjacency list, Min-Heap Dijkstra, HashSet processed tracking, predecessor-based route reconstruction and a version-stamped route cache. This choice is supported by the sparse network structure, the need for frequent dynamic updates, the 16/16 functional test result and the measured few-millisecond uncached routing time on the verified configuration.

## 7. Broader Considerations

### Sustainability and Environment

- SDG 9 - Industry, Innovation and Infrastructure: the design demonstrates resilient infrastructure software that remains responsive as road conditions change.

- SDG 11 - Sustainable Cities and Communities: the system supports disaster-aware evacuation, rerouting and explicit identification of unreachable neighbourhoods.
- SDG 13 - Climate Action: adaptive routing is relevant to disruptions caused by floods, storms, fires and other climate-related emergencies.

## **Society and Safety**

The safety-oriented behaviour of the prototype is to reject invalid inputs, skip blocked roads and report unreachable destinations explicitly rather than returning a misleading route. This is important in an emergency-routing context because a mathematically invalid or stale route should not be presented as usable guidance.

## **Ethics, Accessibility and Professional Responsibility**

The report distinguishes a course prototype from a deployed emergency system. It uses synthetic data and does not claim real-world disaster validation. A future deployment would require validated map data, accessible user interfaces, reliable live updates, fail-safe operation and review by relevant authorities. The current console interface is useful for technical verification but is not an accessibility-complete public interface.

## **8. Conclusion**

The proposed solution models the city as a HashMap-based adjacency list and answers evacuation queries with binary Min-Heap Dijkstra, a HashSet of finalized intersections, predecessor-based route reconstruction and a version-stamped route cache, implemented as modular C.

The major findings are that all 16 functional tests pass; an uncached query on the 8,200-intersection, 21,000-road city averages about 3.91 ms with a worst observed value of 7.000 ms; a cached hit averages about 0.00003 ms; and blocking exactly 6,300 roads still leaves 8,026 of 8,200 intersections reachable while 174 become isolated and route costs rise by +3 to +175.

The assignment requirements are met through dynamic road updates, Min-Heap Dijkstra, HashSet tracking, route reconstruction, versioned caching, validation, complexity analysis, performance measurement and 30% road-failure testing. The main limitations are the single scalar road cost, global cache invalidation, single-threaded execution, clock resolution and synthetic-data evaluation. Possible improvements include A\* or bidirectional search, finer-grained cache invalidation, multi-criteria routing, real road-map import and a user-facing accessible interface.

## 9. Student Reflection

### What I learned beyond the classroom

I learned that choosing an ADT is not only a theoretical complexity exercise; it directly changes whether the whole system scales. The sparse adjacency list avoids the memory cost of a matrix, the Min-Heap changes Dijkstra frontier selection from a linear scan to logarithmic operations, the HashSet makes cyclic-graph processing predictable, and graph-versioned caching shows how data-structure correctness must be maintained when the underlying graph changes. Running the tests and benchmark also showed me why an algorithmic claim should be supported by measured output rather than only by Big-O notation.

### What I would improve with more time or resources

With additional time, I would import real road-map data, separate travel time from safety or risk into a multi-criteria cost model, compare Dijkstra against A\* and bidirectional search, and replace global cache invalidation with dependency-aware invalidation. I would also use a higher-resolution timing method, add concurrent query handling, and build an accessible graphical interface suitable for non-technical users. These changes would make the prototype closer to a realistic smart-city routing service while preserving the core data-structure design.

## 10. References

1. CSA0312 assignment problem statement and assessment rubric (course handout).
2. C language and standard-library concepts used for structures, pointers, arrays, dynamic memory and file I/O.
3. GCC compiler and Windows Command Prompt used as the build and execution environment.
4. GitHub repository and version control: <https://github.com/athisayaprema2007-hue/CSA0312-Intelligent-Evacuation-Routing-System>
5. No external road dataset was used; the full-scale network is deterministic synthetic data generated by the submitted program.
6. ChatGPT and Claude were used as AI-assisted tools for drafting, organization, formatting and review support. Technical results, test outputs and benchmark values reported in this assignment were verified through execution of the submitted program.



## Assessment Rubrics

| Criteria (CO Mapping)                              | Max Marks | Excellent                                                                                             | Good                                               | Satisfactory                         | Needs Improvement                |
|----------------------------------------------------|-----------|-------------------------------------------------------------------------------------------------------|----------------------------------------------------|--------------------------------------|----------------------------------|
| Problem Understanding & ADT Selection CO2          | 15        | Clearly analyzes disaster-routing requirements and selects appropriate ADTs with strong justification | Most requirements identified with appropriate ADTs | Basic understanding of requirements  | Limited understanding            |
| Graph, HashMap & HashSet Design-CO2/CO4            | 20        | Efficient graph representation with correct HashMap/HashSet integration                               | Correct implementation with minor issues           | Basic implementation                 | Inefficient/incomplete design    |
| Min-Heap & Priority Queue Integration-CO4          | 20        | Correct insert, extract-min, heapify/decrease-key operations and efficient integration                | Mostly correct implementation                      | Priority queue works but inefficient | No effective heap implementation |
| Dijkstra & Route Reconstruction-CO5                | 20        | Correct shortest route, dynamic weights, blocked roads and route reconstruction                       | Correct with minor issues                          | Works for simple cases               | Major algorithmic errors         |
| Performance, Testing & Complexity Analysis-CO4/CO5 | 15        | Comprehensive scalability, complexity and performance analysis                                        | Good analysis with minor omissions                 | Basic testing/analysis               | Poor or missing analysis         |
| Reflection & SDG Relevance-CO2/CO5                 | 10        | Strong technical reflection and clear SDG 9, 11 and 13 connection                                     | Good justification and SDG connection              | Generic reflection                   | Missing/irrelevant reflection    |

## Deliverables

- Pseudocode
- Complete modular C implementation
- Test cases and results
- Complexity analysis
- Dijkstra vs Floyd-Warshall comparison
- Performance analysis
- GitHub repository
- Reflection on design decisions and SDG relevance